

Programmation Orientée Objet (POO) en Python

Chapitre 4: Héritage, Réutilisation du Code et Polymorphisme, classes abstraites et méthodes spéciales

Pr Amadou Dahirou GUEYE, UAM
L2 MPI & SML Fév-Avril 2025

Héritage

- L'héritage permet de créer des sous-classes (= sous-catégories) d'objets
 - ◆ La sous-classe « hérite » (= récupère) les attributs et méthodes de la super-classe
 - ◆ On parle aussi de « classe fille ou enfant ou dérivée » et de « super classe ou classe mère ou parent ou base »
 - ◆ Permet la réutilisation des attributs et méthodes, et d'éliminer des redondances
- Pour dire qu'une sous classe B hérite d'une classe de base ou classe parente A:

```
>>> class B(A):
```

Héritage

L'héritage est un concept fondamental en Programmation Orientée Objet (POO) qui permet de créer une nouvelle classe à partir d'une classe existante. Cela favorise la réutilisation du code, l'extensibilité et la maintenance des projets logiciels.

Principe de l'Héritage

L'héritage permet à une classe (classe enfant ou classe dérivée) d'acquérir les attributs et méthodes d'une autre classe (classe parent ou classe de base).

Pourquoi utiliser l'héritage ?

- ✓ Évite la duplication du code.
- ✓ Permet d'organiser les classes de manière logique.
- ✓ Facilite la modification et l'extension du code existant.

Exemple simple d'héritage

```
# Définition d'une classe parent
```

```
class Animal:
```

```
    def __init__(self, nom):  
        self.nom = nom
```

```
    def crier(self):  
        return "Je crie"
```

```
# Définition d'une classe enfant qui hérite d'Animal
```

```
class Chien(Animal):
```

```
    def crier(self):  
        return "J'aboie"
```

Exemple simple d'héritage

```
# Définition d'une autre classe enfant
class Chat(Animal):
    def crier(self):
        return "Miaou Miaou"

# Utilisation
chien = Chien("Rex")
chat = Chat("Minou")

print(chien.nom, ":", chien.crier()) # Rex : Ouaf Ouaf
print(chat.nom, ":", chat.crier())  # Minou : Miaou Miaou
```

- ◆ Ici, **Chien** et **Chat** héritent de la classe **Animal** et redéfinissent la méthode `crier()`

Héritage

Il existe deux types d'héritages:

- Héritage simple
- Héritage multiple

Héritage Simple

Une classe enfant hérite d'une seule classe parent. C'est le cas le plus courant.

```
class Vehicule:  
    def __init__(self, marque):  
        self.marque = marque  
  
    def afficher_info(self):  
        return f"Marque : {self.marque}"
```

Héritage Simple

```
class Voiture(Vehicule):  
    def __init__(self, marque, vitesse):  
        super().__init__(marque) # Appel du constructeur de la classe  
parent  
        self.vitesse = vitesse  
  
    def afficher_info(self):  
        return f"{super().afficher_info()}, Vitesse : {self.vitesse} km/h"
```

Héritage Simple

```
# Utilisation  
voiture = Voiture("Toyota", 180)  
print(voiture.afficher_info()) # Marque : Toyota, Vitesse : 180 km/h
```

◆ Voiture hérite de Vehicule et ajoute un nouvel attribut vitesse. Elle redéfinit aussi la méthode afficher_info() en appelant celle du parent avec super().

Héritage Multiple

Une classe enfant peut hériter de plusieurs classes parents.

```
class Moteur:
    def type_moteur(self):
        return "Moteur essence"

class Carrosserie:
    def type_carrosserie(self):
        return "Carrosserie en métal"
```

Héritage Multiple

```
class Voiture(Moteur, Carrosserie):  
    pass # La classe hérite simplement des deux autres  
  
voiture = Voiture()  
print(voiture.type_moteur())    # Moteur essence  
print(voiture.type_carrosserie()) # Carrosserie en métal
```

Surcharge et Redéfinition de Méthodes avec `super()`

En Programmation Orientée Objet (POO), l'héritage permet aux classes enfants de récupérer les méthodes et attributs d'une classe parent. Cependant, il arrive qu'on veuille modifier ou ajouter du comportement aux méthodes héritées. C'est là qu'interviennent la redéfinition de méthodes et la surcharge avec `super()`.

Redéfinition d'une méthode

La redéfinition de méthode consiste à écrire dans la classe enfant une nouvelle version d'une méthode existante dans la classe parent.

Exemple simple de redéfinition

```
# Définition d'une classe parent
```

```
class Animal:
```

```
    def __init__(self, nom):
```

```
        self.nom = nom
```

```
    def crier(self):
```

```
        return "Je crie"
```

```
# Définition d'une classe enfant qui hérite d'Animal
```

```
class Chien(Animal):
```

```
    def crier(self):
```

```
        return "J'aboie "
```


Exemple simple de redéfinition

```
# Définition d'une autre classe enfant
```

```
class Chat(Animal):  
    def crier(self):  
        return "Miaou Miaou"
```

```
# Utilisation
```

```
chien = Chien("Rex")
```

```
chat = Chat("Minou")
```

```
print(chien.nom, ":", chien.crier()) # Rex : Je crie
```

```
print(chat.nom, ":", chat.crier()) # Minou : Miaou Miaou
```

Exemple simple de redéfinition

Explication :

- ◆ `crier()` existe déjà dans `Animal`, mais `Chien` et `Chat` la redéfinissent pour fournir un comportement spécifique.
- ◆ Quand on appelle `chien.crier()`, la version de `Chien` est utilisée au lieu de celle de `Animal`.

La méthode `super()`

Quand on redéfinit une méthode, on peut vouloir réutiliser la version du parent tout en ajoutant des comportements spécifiques. C'est là qu'on utilise `super()`.

Exemple : Ajouter du comportement avec super()

```
class Personne:
    def __init__(self, nom, age):
        self.nom = nom
        self.age = age

    def se_presenter(self):
        return f"Je suis {self.nom}, j'ai {self.age} ans."
```

Exemple : Ajouter du comportement avec super()

```
class Etudiant(Personne):  
    def __init__(self, nom, age, filiere):  
        super().__init__(nom, age) # Appel du constructeur de Personne  
        self.filiere = filiere  
  
    def se_presenter(self):  
        return f"{super().se_presenter()} Je suis étudiant en {self.filiere}."
```

Exemple : Ajouter du comportement avec `super()`

```
# Utilisation
etudiant = Etudiant("Ali", 22, "Informatique")
print(etudiant.se_presenter())
# Je suis Ali, j'ai 22 ans. Je suis étudiant en Informatique.
```

◆ `super().se_presenter()` appelle la méthode du parent pour ne pas dupliquer du code. ◆
Ensuite, on ajoute un comportement spécifique (Je suis étudiant en {self.filiere}).

super() : Héritage multiple

Lorsqu'une classe hérite de plusieurs classes, `super()` permet de gérer l'appel des méthodes de manière ordonnée. Python utilise l'ordre de résolution des méthodes (MRO, Method Resolution Order) pour éviter des conflits.

Exemple : super() avec héritage multiple

```
class A:  
    def dire_bonjour(self):  
        return "Bonjour de A"  
  
class B(A):  
    def dire_bonjour(self):  
        return f"{super().dire_bonjour()} et de B"
```


Exemple : super() avec héritage multiple

```
class C(A):  
    def dire_bonjour(self):  
        return f"{super().dire_bonjour()} et de C"  
  
class D(B, C): # Héritage multiple  
    def dire_bonjour(self):  
        return f"{super().dire_bonjour()} et de D"
```

Exemple : super() avec héritage multiple

```
# Utilisation  
d = D()  
print(d.dire_bonjour())
```

Résultat :

Bonjour de A et de C et de B et de D

super() : Héritage multiple

- ◆ Python suit l'ordre MRO (D -> B -> C -> A).
- ◆ super() dans D appelle super() dans B, qui appelle super() dans C, et ainsi de suite.⚠ Sans super(), il faudrait dupliquer le code et gérer manuellement l'ordre des appels !

Différence entre Redéfinition et Surcharge en Python

Concept	Explication
Redéfinition de méthode	Une classe enfant modifie une méthode héritée pour changer son comportement.
Surcharge (super())	Une classe enfant appelle la méthode du parent pour réutiliser du code avant d'ajouter son propre comportement.

Avantages de super()

- ✓ Évite la duplication de code en réutilisant la méthode parent.
- ✓ Facilite la maintenance : si on modifie la méthode parent, les enfants en bénéficient.
- ✓ Gère automatiquement l'ordre des appels en cas d'héritage multiple.

Polymorphisme

- Le polymorphisme permet d'attribuer des comportements différents à des objets dérivant les uns des autres, ou au même objet ou en fonction d'un certain contexte
- Les méthodes des classes parentes peuvent être redéfinies dans les classes filles
- Dans ce cas, la méthode de la classe fille peut appeler celle de la classe parente :

```
class Plat(object) :  
    def __init__(self, nom) :  
        self.nom = nom  
  
class PlatEnSauce(Plat) :  
    def __init__(self, nom, sauce) :  
        super().__init__(nom)  
        self.sauce = sauce
```

Appelle `Plat.__init__()`



- Les appels via `super()` peuvent retourner une valeur
`valeur_de_retour_de_la_classe_parente = super().ma_méthode()`
- `super()` gère le cas de l'héritage multiple

Polymorphisme

■ Exemple : classes de cylindres et de cônes : *Classes dérivées - Polymorphisme*

```
class Cercle(object):
```

```
    def __init__(self, rayon):
        self.rayon = rayon
    def surface(self):
        return 3.1416 * self.rayon**2
```

```
class Cylindre(Cercle):
```

```
    def __init__(self, rayon, hauteur):
        Cercle.__init__(self, rayon)
        self.hauteur = hauteur
    def volume(self):
        return self.surface()*self.hauteur
```

la méthode surface() est héritée de la classe parente class

Polymorphisme

■ Exemple : classes de cylindres et de cônes : *Classes dérivées - Polymorphisme*

```
class Cone(Cylindre):  
    def __init__(self, rayon, hauteur):  
        Cylindre.__init__(self, rayon, hauteur)  
  
    def volume(self):  
        return Cylindre.volume(self)/3  
        # cette nouvelle méthode volume() remplace celle que  
        # l'on a héritée de la classe parente (exemple de polymorphisme)
```

Programme test :

```
cyl = Cylindre(5, 7)  
print("Surf. de section du cylindre =", cyl.surface())  
  
print("Volume du cylindre =", cyl.volume())  
  
co = Cone(5,7)  
print("Surf. de base du cône =", co.surface())  
  
print("Volume du cône =", co.volume())
```


Classes abstraites

- Une classe abstraite est une classe faite pour être dérivée
- Exemples de classe abstraites:
 - la classe Animal
 - La classe Etre vivant
- Après il faudra définir des sous classes qui vont hériter de la sous classe avec ses propres attributs et méthodes

Méthodes spéciales

■ De nombreuses méthodes spéciales sont disponibles en Python, de la forme `__nom_de_méthode__()`

◆ `__init__()` (paramètres...) : constructeur

◆ `__del__()` : destructeur (appelé lorsque l'objet est détruit)

◆ `__repr__()` : retourne une représentation d'un objet sous forme de chaîne de caractères pour afficher au programmeur

◆ `__str__()` : retourne une description d'un objet sous forme de chaîne de caractères pour afficher à l'utilisateur final

- `print(objet)` appelle `__str__()` en priorité (à défaut, `__repr__()`)
- `str(objet)` appelle `__str__()` en priorité (à défaut, `__repr__()`)
- `repr(objet)` appelle `__repr__()` en priorité (à défaut, `__str__()`)
- Dans le terminal Python, `__repr__()` est utilisé en priorité :

```
>>> objet  
<chaîne retournée par repr()>
```

Méthodes spéciales :

`__str__` vs `__repr__`

- Quand on crée une classe, on peut représenter notre classe sous forme d'une chaîne de caractères ou sous forme d'une représentation
- Pour cela on peut utiliser 2 méthodes spéciales : la méthode `__str__` et la méthode `__repr__`

Méthodes spéciales : `__str__`

- La méthode spéciale `__str__` nous permet de représenter notre objet sous forme de chaînes de caractères
- Cette méthode est utilisée si on essaie de convertir notre objet en classe avec la fonction de conversion de la classe string `str`
- La classe `str` permet de créer une instance de chaîne de caractères à partir d'un objet. Elle va utiliser la méthode spéciale `__str__` pour créer la chaîne de caractères

```
>>> class User:
...     def __init__(self, first_name, last_name):
...         self.first_name=first_name
...         self.last_name=last_name
...
>>> dahirou=User("Dahirou", "Gueye")
>>> print(str(dahirou))
<__main__.User object at 0x10b3cadd0>
```

On a le nom de la classe à partir de laquelle notre objet a été créé (on a un objet avec son adresse en mémoire).

Méthodes spéciales : `__str__`

- Si on ajoute la fonction `__str__`, on peut retourner dans un premier temps quelque chose qui n'a rien à voir avec notre classe

```
>>> class User:
...     def __init__(self, first_name, last_name):
...         self.first_name=first_name
...         self.last_name=last_name
...     def __str__(self):
...         return "UADB"
...
```

```
>>> dahirou=User("Dahirou", "Gueye")
```

```
>>> print(str(dahirou))
```

```
UADB
```

```
>>>
```

- Il serait plus intéressant de retourner le nom complet de l'utilisateur, dans ce cas, il faut modifier la fonction `__str__` comme suit :

```
return f"{self.first_name} {self.last_name}"
```

Méthodes spéciales : `__repr__`

- La méthode spéciale `__repr__` permet d'indiquer une chaîne de caractères qui sert de représentation à une classe.
- Cette chaîne de caractères est retournée lors de l'utilisation de la fonction `repr` sur une instance de classe

```
>>> import datetime
>>> now = datetime.datetime.now()
>>> print(repr(now))
datetime.datetime(2025, 3, 26, 11, 2, 57, 22553)
>>>
```

- La chaîne de caractères est destinée aux développeurs et sa représentation permet généralement de recréer l'objet en question
- Le code ci-dessus nous retourne une chaîne de caractères que nous pouvons utiliser pour recréer l'objet `now`

```
>>> import datetime
>>> now = datetime.datetime.now(2025, 3, 26, 11, 2, 57, 22553)
>>> print(now)
```

Méthodes spéciales : `__repr__`

■ Pour utiliser cette méthode, il suffit de la surcharger à l'intérieur d'une classe

class Utilisateur:

```
    def __init__(self, prenom, nom):
```

```
        self.prenom = prenom
```

```
        self.nom = nom
```

```
    def __repr__(self):
```

```
        return "Utilisateur(prenom='{}', nom='{}').format(self.prenom, self.nom)
```

```
utilisateur_01 = Utilisateur(prenom="dahirou", nom="Gueye")
```

```
print(repr(utilisateur_01))
```

■ Là encore, nous pouvons prendre le résultat retourné par `repr` (`Utilisateur(prenom='dahirou', nom='Gueye')`) pour recréer l'objet correspondant.

■ Si vous n'implémentez pas cette méthode dans votre classe, la méthode `__repr__` de l'objet `object` (dont toutes les classes héritent par défaut) sera utilisée.

Méthodes spéciales : `__repr__`

■ Par défaut, la classe (`__main__.Utilisateur`) et l'adresse en mémoire de l'instance (`0x10f5952e0`) sont affichées :

On n'implémente pas la méthode `__repr__`

class Utilisateur:

def `__init__`(self, prenom, nom):

 self.prenom = prenom

 self.nom = nom

`utilisateur_01 = Utilisateur(prenom="Patrick", nom="Smith")`

Quand on utilise repr sur notre instance, la classe et l'adresse en mémoire de l'instance sont affichées

`print(repr(utilisateur_01))` *# <__main__.Utilisateur object at 0x10f5952e0>*

Cette implémentation provient de la classe object

- `print(object.__repr__(utilisateur_01))` *# <__main__.Utilisateur object at 0x10f5952e0>*

Méthodes spéciales : `__repr__`

- La méthode spéciale `__repr__` qui permet d'afficher une représentation de ce à quoi doit ressembler notre instance
- Ce qu'on retourne c'est une chaîne de caractères qui nous permettrait de recréer l'objet en question

```
def __repr__(self):  
    return f"User(first_name='{self.first_name}',last_name=  
        '{self.last_name}')
```

```
dahirou=User("Dahirou", "Gueye")
```

- On print après comme suit: `print(repr(dahirou))`
- La chaîne de caractères qui sera retournée après exécution:
`User(first_name='dahirou', last_name='Gueye')`
- On pourra copier coller cette chaîne de caractères pour recréer notre instance
`dahirou= User(first_name='dahirou', last_name='Gueye')`
- Et faire un print de dahirou : `print (dahirou)`

Quelques astuces...

- Obtenir la classe d'un objet : `objet.__class__`
- Obtenir le nom d'une classe : `MaClasse.__name__`

◆ Exemple :

```
class Machine(object):  
    def __repr__(self):  
        return "<%s>" % self.__class__.__name__
```

```
class Ordinateur(Machine):  
    pass
```

```
>>> o = Ordinateur()
```

```
>>> o  
<Ordinateur>
```